

Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 3 September 2014 - Part I: Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY

COMMENTED - THE CANDIDATE SHOULD CORRECTLY ANSWER AND REASONABLY EXPLAIN WHY

NAME:

MATRICOLA:

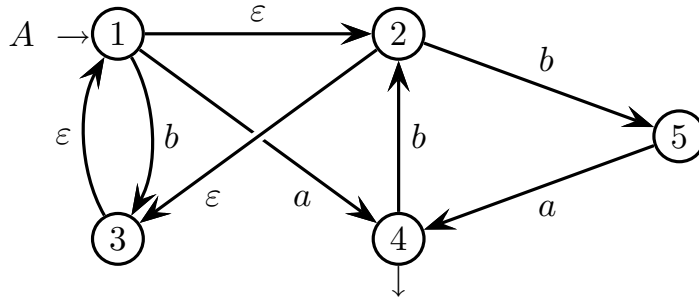
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the following nondeterministic finite state automaton A , over the alphabet $\{a, b\}$ and with spontaneous transitions (ε -transitions):

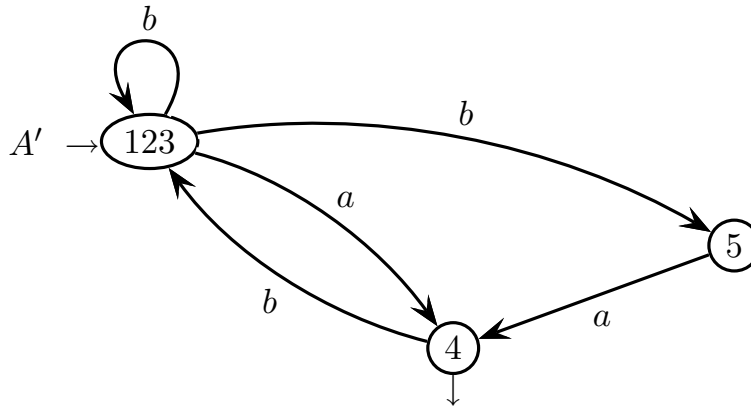


Answer the following questions:

- (a) Transform the automaton A into an equivalent automaton A' without spontaneous transitions (ε -transitions), no matter if it is still nondeterministic.
 - (b) If the automaton A' obtained before is nondeterministic, then in a systematic way of your choice transform it into an equivalent deterministic automaton A'' , and if necessary minimize it.
 - (c) In a systematic way of your choice, find a regular expression R that generates the (regular) language $L(A)$.
 - (d) (optional) Say if the language $L(A)$ is local or not, and explain why; if such a language is local, then construct the local automaton A''' that recognizes it.
-

Solution

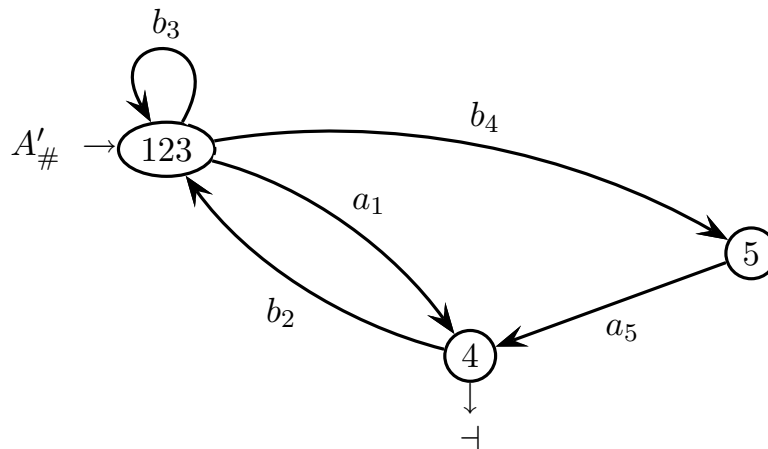
- (a) The only reason why automaton A is nondeterministic, is that it has three spontaneous transitions. We easily see that these spontaneous transitions make a cyclic path, namely $1 \xrightarrow{\varepsilon} 2 \xrightarrow{\varepsilon} 3 \xrightarrow{\varepsilon} 1$. By collapsing the spontaneous cycle (ε -loop), we obtain the following automaton A' :



Automaton A' does not have any more spontaneous transitions, yet it is not deterministic as state 123 has two outgoing b -transitions.

Of course, we could remove the spontaneous transitions one by one, by repeatedly using the suited procedure for cutting an ε -transition. The result would be equivalent to collapsing the spontaneous cycle, though not necessarily identical.

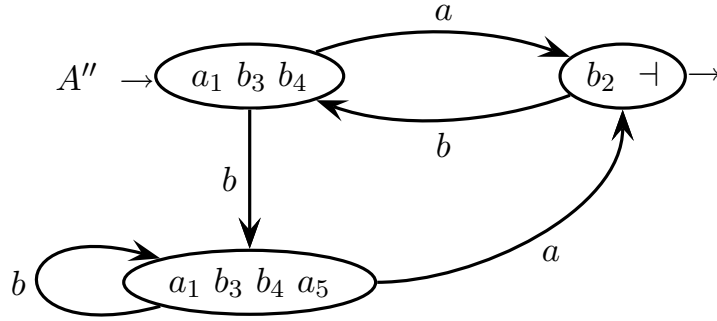
- (b) To determinize and minimize the automaton, we just need to apply the Berry-Sethi algorithm and the tabular minimization algorithm, respectively. Here is how to proceed. Start from automaton A' , number its transitions and also give it an end-marker, and thus obtain a numbered and marked automaton $A'_{\#}$:



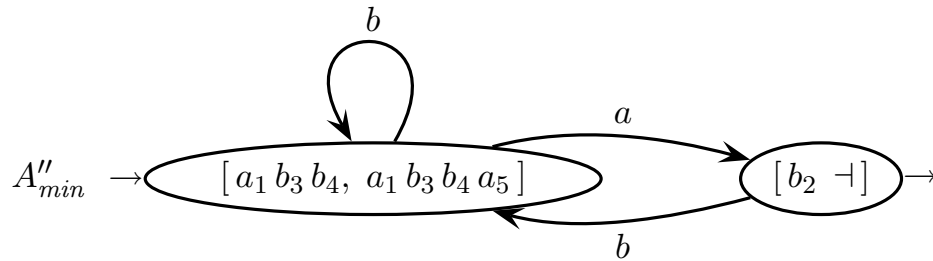
Next, compute the initials and the followers of automaton $A'_\#$:

<i>initials</i>	$a_1 \ b_3 \ b_4$
<i>generators</i>	<i>followers</i>
a_1	$b_2 \ \neg$
b_2	$a_1 \ b_3 \ b_4$
b_3	$a_1 \ b_3 \ b_4$
b_4	a_5
a_5	$b_2 \ \neg$

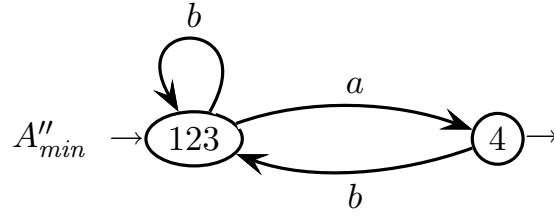
Next, design the deterministic automaton A'' :



Finally, minimize the automaton A'' . We could apply the full tabular minimization algorithm, but here we can save work and do more simply though still systematically. In fact, only the two non-final states might be undistinguishable, and actually they are: their two a -transitions are directed to the same destination state, and their two b -transitions are also directed to the same destination state. Thus by merging the two non-final states into one equivalence class, and by leaving the final state isolated, we obtain the minimal form A''_{min} of A'' :

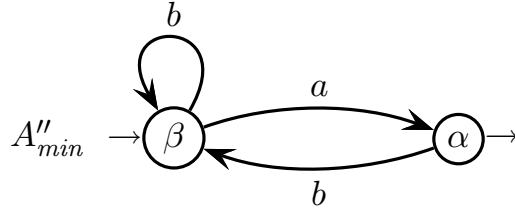


Although, by directly eliminating the redundant path $123 \xrightarrow{b} 5 \xrightarrow{a} 4$ from automaton A' , as such a path does the same recognition work as path $123 \xrightarrow{b} 123 \xrightarrow{a} 4$ does, we directly get this form:

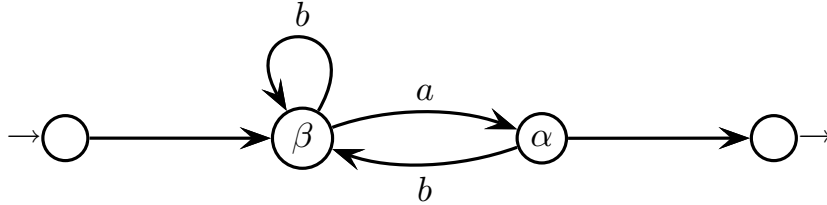


which is deterministic and clearly minimal, because its two states are trivially distinguishable as one is final and the other is not. Therefore it must be isomorphic to automaton A''_{min} , as it is immediate to see.

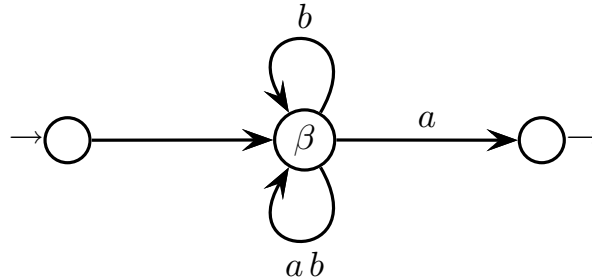
- (c) To find an equivalent regular expression R , we just need to apply the node elimination algorithm (Brzozowski) to automaton A , A' or A'' (in the minimal form). Clearly here the most convenient choice is A''_{min} as it has fewer nodes, so we proceed with it (and for brevity we rename the states):



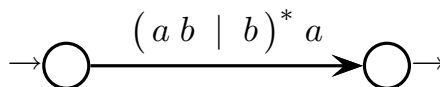
Add an initial / a final state without ingoing / outgoing arcs:



Eliminate state α :



Eliminate state β and thus obtain one regular expression:



Eventually we have the following regular expression R that recognizes the language $L(A''_{min}) = L(A)$:

$$R = (a b \mid b)^* a$$

Although, again by starting directly from automaton A''_{min} , but now by closely examining the paths to the final state and from it back to the initial one, we can intuitively obtain the following regular expression R' , different from R :

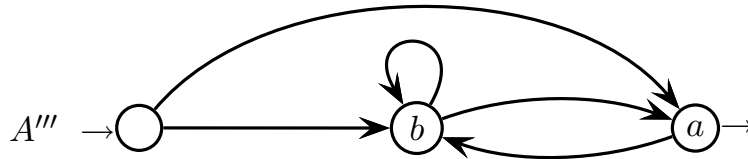
$$R' = b^* a (b^+ a)^*$$

The two expressions R and R' are equivalent by construction. To check it is so, start from the latter and transform it by some well known regular identity:

$$\begin{aligned}
R' &= b^* a (b^+ a)^* \\
&= b^* a \underbrace{b^+ a b^+ \dots a b^+ a}_{n \geq 0 \text{ times}} \\
&= b^* \underbrace{a b^+ a b^+ \dots a b^+}_{n \geq 0 \text{ times}} a \\
&= b^* (a b^+)^* a \\
&= b^* (a b \cdot b^*)^* a && \text{remember that } \beta^* (\alpha \beta^*)^* = (\alpha \mid \beta)^* \\
&= (a b \mid b)^* a \\
&= R
\end{aligned}$$

Of course, there may be other regular expressions equivalent to R and R' .

- (d) The deterministic automaton A''_{min} obtained before recognizes a local language, because every state of A''_{min} is entered by transitions with only one label type. Thus the original language $L(A) = L(A''_{min})$ is local, too. We obtain the local automaton A''' , which must be deterministic as well, by adding to automaton A''_{min} an initial state without ingoing arcs and by labeling the states (not the arcs) with the ingoing character (the initial state is left unlabeled). Here it is:



Obviously by construction the local automaton A''' is in the minimal form, as the deterministic automaton A''_{min} , from which it is obtained, is already minimal.

2 Free Grammars and Pushdown Automata 20%

1. Consider a subset L of the Dyck language with round brackets ‘(’ and ‘)’’, where the strings of L contain an even number of open round brackets (number 0 is even).

Sample valid strings:

$(()) \quad ((())()) \quad ()((())())()$

Sample invalid strings:

$() \quad ((())() \quad ((())()$

Answer the following questions:

- (a) Write a *BNF* grammar G , not ambiguous, that generates the language L .
 - (b) (optional) Draw the syntax trees of grammar G for the three sample valid strings.
-

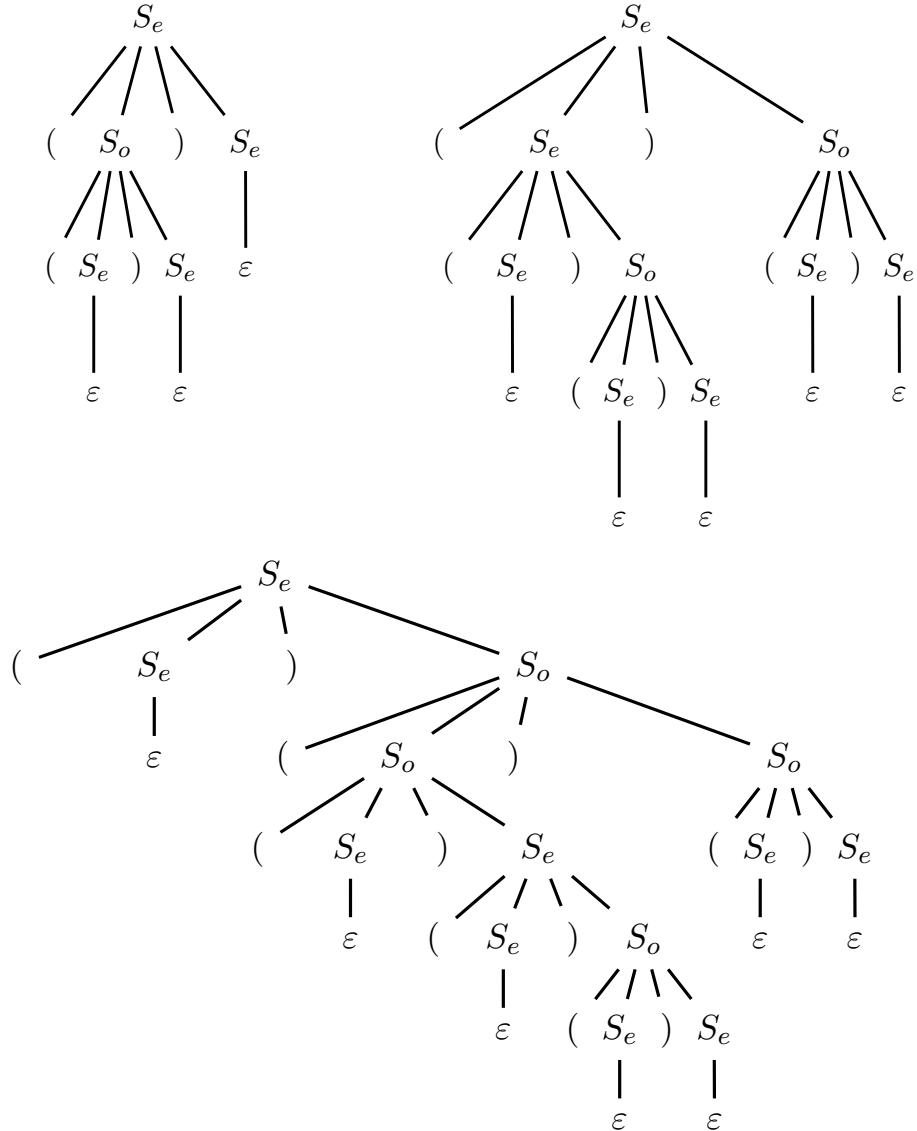
Solution

(a) Here is grammar G (axiom S_e):

$$G \begin{cases} S_e \rightarrow ' (' S_o ') ' S_e \mid ' (' S_e ') ' S_o \mid \varepsilon \\ S_o \rightarrow ' (' S_e ') ' S_e \mid ' (' S_o ') ' S_o \end{cases}$$

The nonterminals S_e and S_o generate strings with an even and odd number of bracket pairs, respectively. Notice the sublanguages $L(S_e)$ and $L(S_o)$ are disjoint, i.e., $L(S_e) \cap L(S_o) = \emptyset$. Grammar G is easily obtained from the standard *BNF* Dyck rule $S \rightarrow (S) S \mid \varepsilon$, which is not ambiguous, by splitting the nonterminal S into two ones, namely S_e and S_o , that generate disjoint sublanguages, as seen before; thus grammar G is not ambiguous either.

(b) Here are the three sample syntax trees of grammar G :



2. One wishes one modeled the fragment of a programming language, similar yet not identical to the C language, that has the variable assignment statement and the n -way conditional if statement (sometimes called *chained if*), with $n \geq 1$. The n -way if conditional has an if-then way, a number ≥ 0 of else-if-then ways (each such way has its own condition) and an optional final else way (unconditioned). The new language closes the whole n -way if conditional with exactly one keyword `endif`.

The following specifications apply:

- The variable assignment statement has a variable name on the left side, the operator '=' in the middle and a numerical expression on the right side.
- The numerical expression has variables, constants, round brackets, and the operators of addition '+', subtraction '-', and multiplication '*', with the usual precedences (addition and subtraction are lower priority than multiplication).
- The condition of an if is a relational expression with a comparison operator '<', '==' or '>' between two numerical expressions (as defined before).
- The then way of an if is mandatory, the else if and else ways are all optional.
- There may be statement blocks (not empty and possibly nested), which are delimited by graph brackets '{' e '>'; one isolated statement in a then or else way does not need to have graph brackets around itself (but it may have them).
- Two consecutive statements in a block must be separated by a semicolon ';'.
- It is forbidden to have a semicolon ';' ahead of the closed graph bracket '}', as well as ahead of the keywords `else` (possibly followed by an if) and `endif`.

Here are a sample 3-way if conditional and a sample 2-way (ordinary) if conditional:

```

{
    a = b + c * (a + 2) ;      /* language phrase      */
    {                          /* assignment          */
        a = 1 + c ;           /* nested block      */
        c = b * a
    } ;
    if a - b > c                /* 3-way if: 1-st way */
        b = 2
    else if 2 * (b + c) < 3     /* 3-way if: 2-nd way */
        a = 1
    else {                      /* 3-way if: 3-rd way */
        if b == c {            /* 2-way if: 1-st way */
            c = 3
        } else                 /* 2-way if: 2-nd way */
            a = 2 * b
        endif ;                /* end of the 2-way if */
        c = - c                 /* unary minus sign   */
    } endif                     /* end of the 3-way if */
}

```

A language phrase consists of exactly one statement block. Write an *EBNF* grammar, not ambiguous, that generates the language fragment described above.

Solution

Here is a grammar that fulfills the specifications (axiom PROG):

$\langle \text{PROG} \rangle$	$\rightarrow$	$\langle \text{BLOCK} \rangle$
$\langle \text{BLOCK} \rangle$	$\rightarrow$	$\{ \langle \text{STLST} \rangle \}$
$\langle \text{STLST} \rangle$	$\rightarrow$	$(\langle \text{STAT} \rangle ;)^* \langle \text{STAT} \rangle$
$\langle \text{STAT} \rangle$	$\rightarrow$	$\langle \text{ASGN} \rangle \mid \langle \text{NWAYIF} \rangle \mid \langle \text{BLOCK} \rangle$
$\langle \text{ASGN} \rangle$	$\rightarrow$	$\langle \text{VID} \rangle = \langle \text{NEXP} \rangle$
$\langle \text{NWAYIF} \rangle$	$\rightarrow$	if $\langle \text{REXP} \rangle \langle \text{STAT} \rangle \langle \text{ELSIF} \rangle^* [\langle \text{ELSE} \rangle]$ endif
$\langle \text{ELSIF} \rangle$	$\rightarrow$	else if $\langle \text{REXP} \rangle \langle \text{STAT} \rangle$
$\langle \text{ELSE} \rangle$	$\rightarrow$	else $\langle \text{STAT} \rangle$
$\langle \text{REXP} \rangle$	$\rightarrow$	$\langle \text{NEXP} \rangle (< \mid == \mid >) \langle \text{NEXP} \rangle$
$\langle \text{NEXP} \rangle$	$\rightarrow$	$[-] \langle \text{TERM} \rangle ((+ \mid -) \langle \text{TERM} \rangle)^*$
$\langle \text{TERM} \rangle$	$\rightarrow$	$\langle \text{FACT} \rangle (* \langle \text{FACT} \rangle)^*$
$\langle \text{FACT} \rangle$	$\rightarrow$	$\langle \text{VID} \rangle \mid \langle \text{NUM} \rangle \mid (\langle \text{NEXP} \rangle)$
$\langle \text{VID} \rangle$	$\rightarrow$	...
$\langle \text{NUM} \rangle$	$\rightarrow$	...

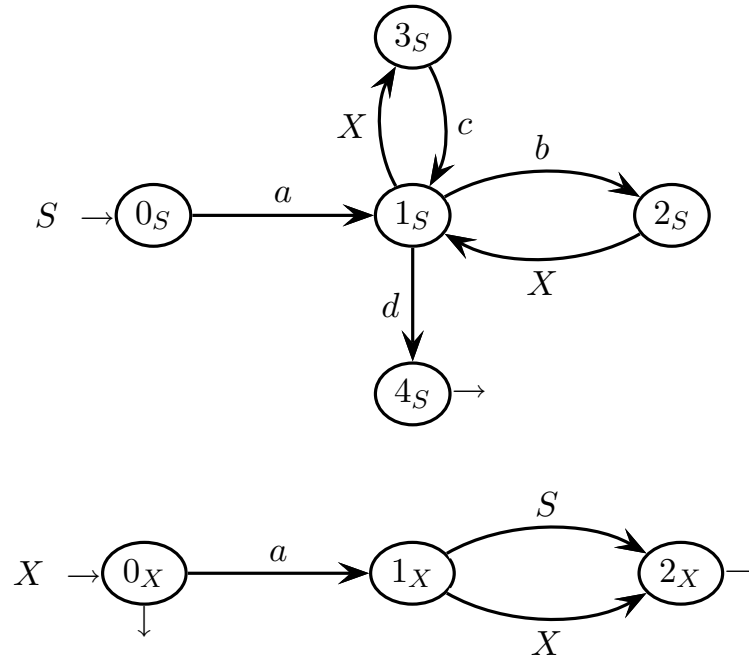
Just for clarity, the names of the syntactic classes read this way: program (PROG), statement block (BLOCK), statement list (STLST), statement (STAT), assignment statement (ASGN), n -way if conditional statement (NWAYIF), else if way (ELSIF), else way (ELSE), relational expression (REXP), numerical expression (NEXP), numerical term (TERM), numerical factor (FACT), variable identifier (VID) and number (NUM).

This grammar is *EBNF*, and it is not ambiguous by construction (furthermore it would not be difficult to verify it is *ELL*(2)). The square brackets indicate optionality. The nonterminals VID and NUM generate a variable identifier and a number, respectively, and here they are left unexpanded. The nonterminal NWAYIF generates the n -way if conditional and works correctly for any number $n \geq 1$ of ways. The first way (i.e., the **then** branch of the if) is mandatory. The nonterminal ELSIF generates a way with a condition and a statement (but no closing **endif**); there may be none, one or more such conditioned ways. The optional nonterminal ELSE generates the final unconditioned way with a statement. In the case this nested last final statement is an if conditional itself, it has its own closing **endif**, so that it does not cause any ambiguity with the else if ways that may precede it (which are closed altogether by the final **endif**). The NWAYIF rule finishes by generating the keyword **endif**, which closes the whole n -way if. The rest of the grammar is somewhat standard and deserves no special comment.

11

3 Syntax Analysis and Parsing 20%

1. Consider the following recursive machine network $\mathcal{M}$, over the terminal alphabet $\{a, b, c, d\}$ and the nonterminal alphabet $\{S, X\}$ (axiom S):



Answer the following questions:

- (a) Construct a part of the pilot $\mathcal{P}$ of net $\mathcal{M}$ sufficient to parse the following valid string, according to the bottom-up analysis method (i.e., method *ELR*):

$a b d$

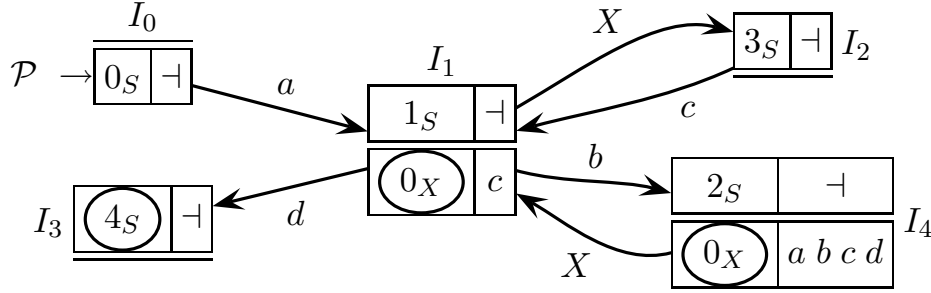
- (b) By using the pilot part $\mathcal{P}$ constructed before, simulate the bottom-up parsing process of the valid string “ $a b d$ ” considered above. You should show move-by-move the evolution of the parser stack, list one-by-one the shift and reduction moves the parser executes, and draw the syntax tree the parser builds. Please fill and complete the simulation table prepared on the next page.
- (c) Examine the condition *ELL*(1) for net $\mathcal{M}$ by computing all the guide sets on the arcs and final darts of net $\mathcal{M}$, say if the net $\mathcal{M}$ is *ELL*(1) and explain why.
- (d) (optional) The net $\mathcal{M}$ is *not* of type *ELR*(1). Continue (if it is necessary) the construction of the pilot $\mathcal{P}$ of net $\mathcal{M}$, as much as it helps to find a conflict (which has to exist). Then find a valid string, i.e., a string of $L(\mathcal{M})$, that is *nondeterministically* recognized by pilot $\mathcal{P}$, and briefly explain why this happens.

simulation table of the parsing process of string “ $a b d$ ” to be completed
 (the number of rows is not significant)

stack base	→ stack contents →							move executed
$\langle 0_S, \neg, \perp \rangle$	empty initial stack							initialization
	a	$\langle 1_S, \neg, \#1 \rangle$ $\langle \textcircled{0_X}, c, \perp \rangle$						terminal shift $I_0 \xrightarrow{a} I_1$

Solution

- (a) Here is a partial pilot $\mathcal{P}$ of network $\mathcal{M}$ with five m-states, which is (more than) sufficient to simulate the recognition of string “ $a b d$ ”:



The m-states I_1 and I_4 have outgoing a -transitions as well, which connect them to quite a few more m-states that here are omitted.

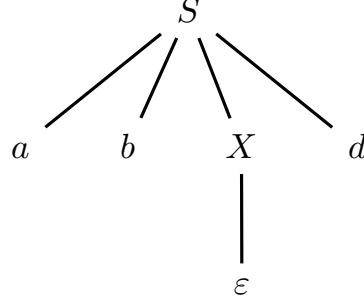
The pilot $\mathcal{P}$ drives the parser to recognize string $a b d \in L(\mathcal{M})$ through path $I_0 \xrightarrow{a} I_1 \xrightarrow{b} I_4 \xrightarrow{X} I_1 \xrightarrow{d} I_3$. The nonterminal shift $I_4 \xrightarrow{X} I_1$ is caused by the null reduction $\varepsilon \rightsquigarrow X$ that originates in the m-state I_4 from the item $\langle 0_X, a b c d \rangle$ with final state 0_X and look-ahead character d (out of the four possible ones).

- (b) Here is the complete simulation of the analysis of the valid string “ $a b d$ ”:

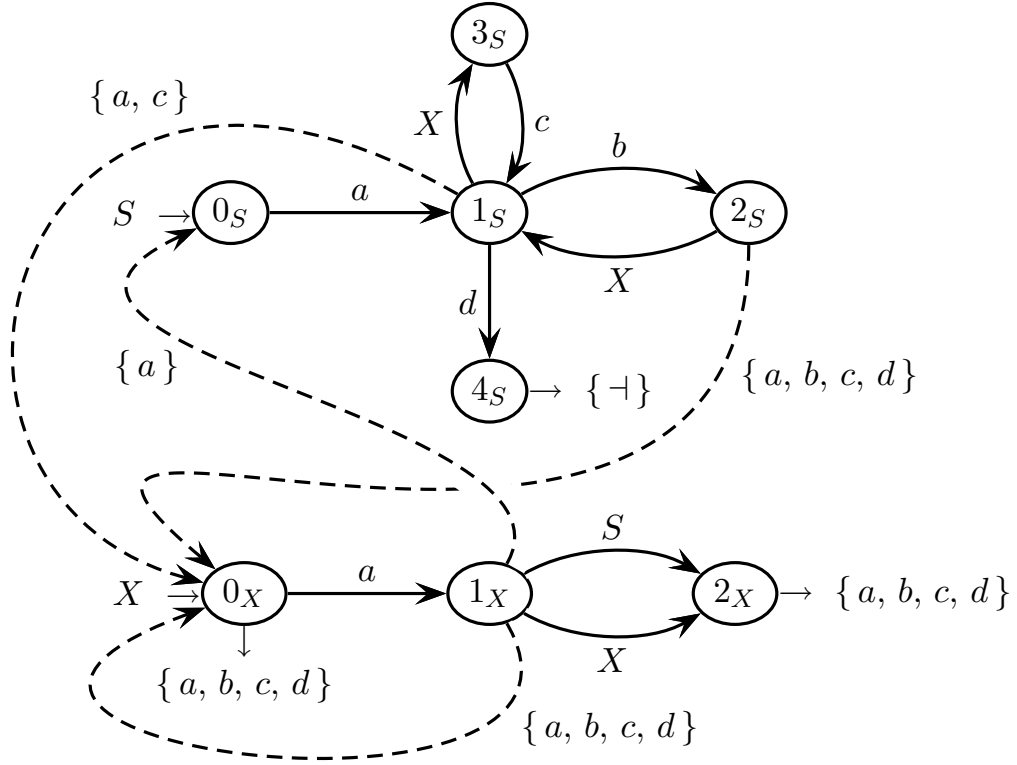
stack base	→ stack contents →				move executed
$\langle 0_S, \perp, \perp \rangle$	empty initial stack				initialization
	a	$\langle 1_S, \perp, \#1 \rangle$ $\langle 0_X, c, \perp \rangle$			terminal shift $I_0 \xrightarrow{a} I_1$
	a	$\langle 1_S, \perp, \#1 \rangle$ $\langle 0_X, c, \perp \rangle$	b	$\langle 2_S, \perp, \#1 \rangle$ $\langle 0_X, abcd, \perp \rangle$	terminal shift $I_1 \xrightarrow{b} I_4$
	a	$\langle 1_S, \perp, \#1 \rangle$ $\langle 0_X, c, \perp \rangle$	b	$\langle 2_S, \perp, \#1 \rangle$ $\langle 0_X, abcd, \perp \rangle$	null reduction $\varepsilon \rightsquigarrow X$
	a	$\langle 1_S, \perp, \#1 \rangle$ $\langle 0_X, c, \perp \rangle$	b	$\langle 2_S, \perp, \#1 \rangle$ $\langle 0_X, abcd, \perp \rangle$	nonterminal shift $I_4 \xrightarrow{X} I_1$
	a	$\langle 1_S, \perp, \#1 \rangle$ $\langle 0_X, c, \perp \rangle$	b	$\langle 2_S, \perp, \#1 \rangle$ $\langle 0_X, abcd, \perp \rangle$	X
	a	$\langle 1_S, \perp, \#1 \rangle$ $\langle 0_X, c, \perp \rangle$	b	$\langle 2_S, \perp, \#1 \rangle$ $\langle 0_X, abcd, \perp \rangle$	X
	a	$\langle 1_S, \perp, \#1 \rangle$ $\langle 0_X, c, \perp \rangle$	b	$\langle 2_S, \perp, \#1 \rangle$ $\langle 0_X, abcd, \perp \rangle$	X
empty final stack					reduction $a b X d \rightsquigarrow S$
					stop & accept

After move number 6 (initialization is move number 0), it happens that: (i) the stack is empty (i.e., it contains only the base stack symbol), (ii) the pilot is back to m-state I_0 , (iii) the input is completely consumed, and (iv) the last operation is a reduction to the axiom S . So the acceptance condition is verified. Notice that the last nonterminal shift on S (that officially should immediately follow the reduction to S) does not take place as the parser has stopped.

The two reductions define the following syntax tree of the sample string “ $ab d$ ”:

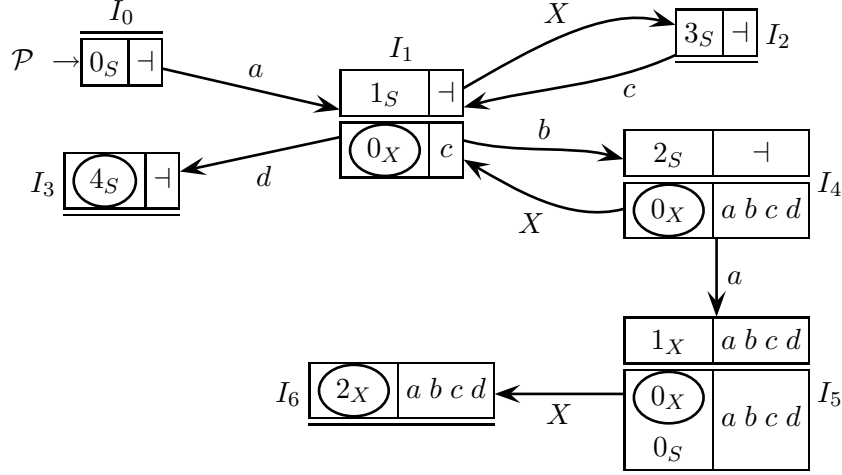


- (c) Here are all the guide sets on the machine network $\mathcal{M}$ completed with the call arcs (the guide sets on the terminal shift arcs are trivial and here are not shown):



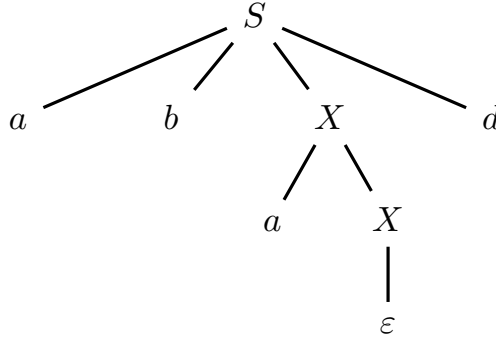
The reader may check the guide sets by using the recursive equations that define them. Remember that the nonterminal X is nullable, while the axiom S is not. The axiomatic machine M_S fits to the $LL(1)$ analysis, yet machine M_X does not. In fact, the former machine has a branching point on state 1_S , and the look-ahead sets of the three outgoing transitions (one call arc and two shift arcs), namely $\{a, c\}$, $\{b\}$ and $\{d\}$, are disjoint. Instead, the latter has a bifurcation point on state 0_X (one shift arc and one final dart) with overlapping look-ahead sets $\{a\}$ and $\{a, b, c, d\}$, and it has another bifurcation point on state 1_X (two call arcs) also with overlapping look-ahead sets $\{a\}$ and $\{a, b, c, d\}$. Therefore the network $\mathcal{M}$ is not of type $ELL(1)$ (actually it is not of type $ELR(1)$ either).

- (d) Here is the partial pilot $\mathcal{P}$ with two more m-states, namely I_5 and I_6 (the m-state I_5 also has an outgoing S -transition, which here is omitted):



Now the partial pilot $\mathcal{P}$ exhibits a shift-reduce conflict on character a in the m-state I_4 , caused by the reduction item $\langle 0_X, a b c d \rangle$ with final state 0_X and look-ahead character a (out of the four possible ones), and by the outgoing a -transition. Therefore the network $\mathcal{M}$ is not of type $ELR(1)$. There may be other conflicts (the reader may wish to draw the full pilot by himself).

The longer string “ $a b a d$ ” also belongs to the language $L(\mathcal{M})$. In fact, the parser executes the following moves: shift $I_0 \xrightarrow{a} I_1 \xrightarrow{b} I_4 \xrightarrow{a} I_5$, reduce $\varepsilon \rightsquigarrow X$ and shift $I_5 \xrightarrow{X} I_6$, reduce $a X \rightsquigarrow X$ (this reduction sends the pilot back to m-state I_4) and shift $I_4 \xrightarrow{X} I_1$, shift $I_1 \xrightarrow{d} I_3$, reduce $a b X d \rightsquigarrow S$, so finally stop and accept. For better clarity, here is the syntax tree of string “ $a b a d$ ”:



However, such a valid string is recognized nondeterministically. In fact, after reading the string prefix “ $a b$ ” the parser is in the m-state I_4 , where it is unable to choose deterministically whether to read the next character a and shift to m-state I_5 (as seen before, this choice will succeed in recognizing the string), or whether to execute a null reduction to X with look-ahead a and immediately shift to m-state I_1 (this choice will eventually fail to recognize the string). There may be longer strings that are recognized nondeterministically.

4 Translation and Semantic Analysis 20%

1. Consider the syntactic translation of the n -way if conditional with `endif`, for $n \geq 3$ (see also ex. 2.2), into the ordinary 2-way if conditional also with `endif`. The n -way if must be translated into a series (of length $n - 1$) of 2-way if's, which are nested inside of their `else` ways and are closed one-by-one with as many keywords `endif`.

Here is the sample translation of a 3-way if into two nested 2-way if's:

if cond_1 /* 1-st way */	if cond_1 /* 1-st way */
block_1	block_1
else if cond_2 /* 2-nd way */	else /* 2-nd way */
block_2	if cond_2 /* 1-st way */
else /* 3-rd way */	block_2
block_3	else /* 2-nd way */
endif	block_3
	endif
	endif
source with one 3-way if	translation into two nested 2-way if's

The translation emits an output different from the input if the source if conditional has three or more ways. If the source if conditional is 2-way or 1-way only, then the translation outputs it unchanged. Suppose the condition and the block are generated by the terminals C and B , respectively, which do not need to be expanded.

Here is the source *BNF* grammar G_s (axiom S), which generates the n -way if ($n \geq 1$):

$$G_s \left\{ \begin{array}{ll} S & \rightarrow \text{if } C \ B \ S_{else} \\ S_{else} & \rightarrow \text{else if } C \ B \ S_{else} \\ S_{else} & \rightarrow \text{else } B \ \text{endif} \\ S_{else} & \rightarrow \text{endif} \end{array} \right.$$

Answer the following questions:

- (a) Write the destination grammar G_d that corresponds to the source grammar G_s , for the translation described above (the translation must work for every $n \geq 1$).
- (b) Draw the source and destination syntax trees of the sample translation shown above (consider the symbols C and B as terminals).
- (c) (optional) Examine the translation scheme (or grammar) written before, say if it is deterministic or not, and briefly explain why.

Solution

- (a) In a schematic form, suppose that the if-then way is represented by i , an else way by e (with or without an if), the final keyword **endif** by f and a block by b . So a generic source string is $ib(eib)^n[eb]f$ and the corresponding translated string is $ib(eib)^n[eb]f^{n+1}$ (with $n \geq 0$). Even more schematically, by dropping the header ib and the optional substring eb , and by compacting the substring eib into a character a , the translation core becomes $a^n f \mapsto a^n f^{n+1}$. Clearly this core is syntactic, as it simply translates an iterative structure into a nested one. Here is a viable destination grammar G_d (axiom S), which simply adds a keyword **endif** to each else if way, and of course keeps for the whole if the final keyword **endif** that is already generated by the source grammar G_s :

$$G_d \left\{ \begin{array}{ll} S & \rightarrow \text{if } C B S_{else} \\ S_{else} & \rightarrow \text{else if } C B S_{else} \text{ endif} \\ S_{else} & \rightarrow \text{else } B \text{ endif} \\ S_{else} & \rightarrow \text{endif} \end{array} \right.$$

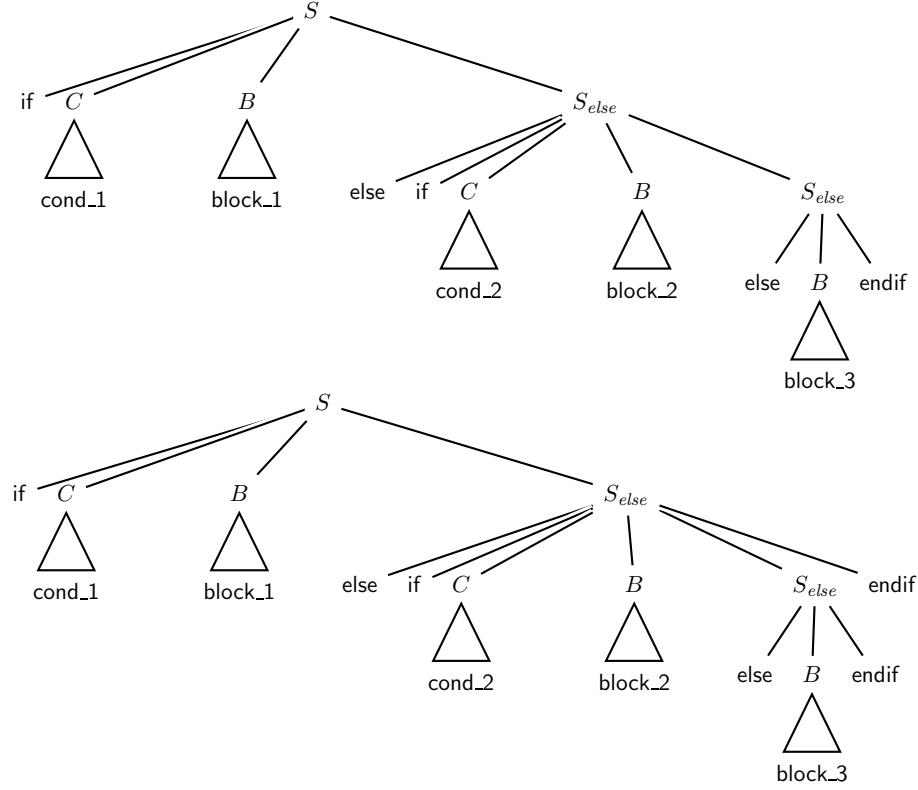
Grammar G_d has the same (nonterminal) structure as grammar G_s , so altogether they make a syntactic translation scheme, which works as specified for every $n \geq 1$ number of ways. Notice that for the two limit cases $n = 1, 2$ the translated text is identical to the source one, as prescribed. Notice also that the source language is regular, whereas the destination one is not (see C and B as terminals).

For completeness, here is the translation grammar G_τ that corresponds to the translation scheme (axiom S - the target terminals are enclosed in braces):

$$G_\tau \left\{ \begin{array}{ll} S & \rightarrow \text{if } \{\text{if}\} C B S_{else} \\ S_{else} & \rightarrow \text{else } \{\text{else}\} \text{if } \{\text{if}\} C B S_{else} \{\text{endif}\} \\ S_{else} & \rightarrow \text{else } \{\text{else}\} B \text{endif } \{\text{endif}\} \\ S_{else} & \rightarrow \text{endif } \{\text{endif}\} \end{array} \right.$$

In this form we can see more directly that the destination has one keyword **endif** for each way **else if**, whereas the source has one **endif** for the whole n -way **if**.

(b) Here are the two source and destination syntax trees of the sample conditional:



The nonterminals C and B are left unexpanded and are represented as generic subtrees that generate some condition and block, respectively.

- (c) It is easy to see that the *BNF* source grammar G_s is of type $LL(2)$. In fact, taken a look-ahead window of size two, the alternative rules of nonterminal S_{else} start with different terminal digrams, provided nonterminal B may not generate any string that starts with a terminal `if`. Such a restriction is at all reasonable, as nonterminal B is supposed to generate a statement block, so it should start with a keyword like `begin`, or a symbol like `{`, or similar ones. In conclusion, the proposed syntactic translation scheme is reasonably deterministic *LL*.

As for the *LR* option, the translation grammar G_τ has to be written with all its target terminals at the rule end, so that a write action may occur only at reduction. Here is a postfix form G'_τ of G_τ with two more auxiliary nonterminals E and I (axiom S - the target terminals are enclosed in graph brackets):

$$G'_\tau \left\{ \begin{array}{ll} S \rightarrow \text{if } I C B S_{\text{else}} & E \rightarrow \{ \text{else} \} \\ S_{\text{else}} \rightarrow \text{else } E \text{ if } I C B S_{\text{else}} \{ \text{endif} \} & I \rightarrow \{ \text{if} \} \\ S_{\text{else}} \rightarrow \text{else } E B \text{ endif } \{ \text{endif} \} & \\ S_{\text{else}} \rightarrow \text{endif } \{ \text{endif} \} & \end{array} \right.$$

Under the assumption that the subgrammars of the nonterminals C and B are themselves of type $LR(1)$, it is easy to see that the source component of grammar G'_τ is of type $LR(1)$. The reader may wish to draw the pilot of the source component of G'_τ and check the $LR(1)$ condition on it. Therefore the proposed translation is reasonably deterministic *LR*.

2. A grammar G generates expressions that consist of a non-empty list delimited by round brackets '(' and ')'. Such a list contains one or more elements of these two types: an atom represented by terminal a or a non-empty sublist delimited by brackets (as before); and so on recursively down to an arbitrary sublist nesting depth.

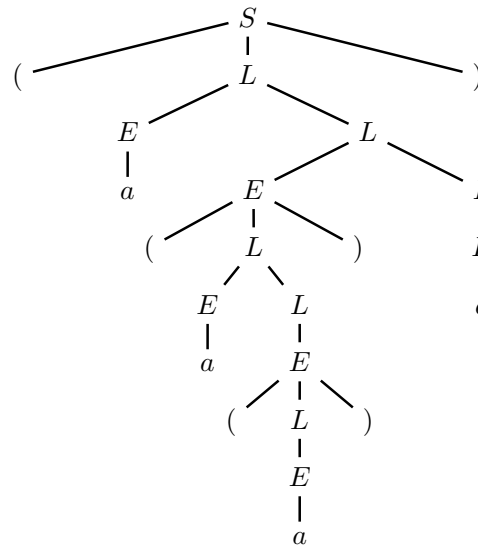
The nesting level of an element (atom or sublist) is the number of bracket pairs that enclose the element. Here is a sample expression e :

$$e = \left(a \left(a \left(a \right) \right) a \right)$$

The expression e has three elements at level 1, i.e., the 1st and 4th atom a , and the sublist $(a(a))$; it has two at level 2, i.e., the 2nd atom a and the sublist (a) ; and only one at level 3, i.e., the 3rd atom a . It has a *total* number of elements $3+2+1 = 6$. The sublist $(a(a))$ has three elements in total: two atoms a and the sublist (a) .

Here is the grammar G (axiom S) of the expressions, and the syntax tree of e :

$$G \left\{ \begin{array}{l} S \rightarrow '(' L ') \\ L \rightarrow E L \\ L \rightarrow E \\ E \rightarrow a \\ E \rightarrow '(' L ') \end{array} \right.$$



Answer the following questions (use the tables and trees on the next pages):

- Write an attribute grammar G_a based on the syntactic support G . Grammar G_a computes an integer attribute $n \geq 1$ that expresses the total number of elements (atoms and sublists) in the expression (i.e., the number of nonterminals E). In the tree root of e it holds $n = 6$. Decorate the tree of e with the values of n .
- By means of an integer attribute $d \geq 1$, associate to each element (atom or sublist) the respective nesting level. Decorate the tree of e with the values of d .
- (optional) By means of a boolean attribute v , and possibly of more ones if they help, verify if in the expression there is a proper sublist (i.e., not coincident with the entire expression) that has a total number of elements equal to its nesting level. If there is, in the tree root it holds $v = T$, otherwise it holds $v = F$. The expression e has $v = F$ in the root. Instead, the expression $e' = (a(a)a)$ (different from e) has $v = T$, because its proper sublist (a) has only one element and is at level 1. Decorate the tree of e with the attribute values.

attributes already assigned to be used for grammar G_a

write the field *type*

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
	n	integer ≥ 1	S, L, E	total number of elements (atoms and sublists)
	d	integer ≥ 1	L, E	nesting level of an element (atom or sublist)
	v	boolean	S, L, E	this predicate is true if and only if in the expression there is a proper sublist (i.e., not coincident with the entire expression) that has a total number of elements (atoms and sublists) equal to its nesting level

possible auxiliary attributes to be added for question c (if they help)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>

syntax

semantics - question *a*

$$\mathbf{n_0} = \mathbf{n_1}$$

$$1: S_0 \rightarrow (L_1)$$

$$2: L_0 \rightarrow E_1 L_2$$

$$3: L_0 \rightarrow E_1$$

$$4: E_0 \rightarrow a$$

$$5: E_0 \rightarrow (L_1)$$

syntax

semantics - question *b*

d₁ = 1

1: $S_0 \rightarrow (L_1)$

2: $L_0 \rightarrow E_1 L_2$

3: $L_0 \rightarrow E_1$

4: $E_0 \rightarrow a$

5: $E_0 \rightarrow (L_1)$

syntax

semantics - question *c*

$$\mathbf{v_0} = \mathbf{v_1}$$

$$1: S_0 \rightarrow (L_1)$$

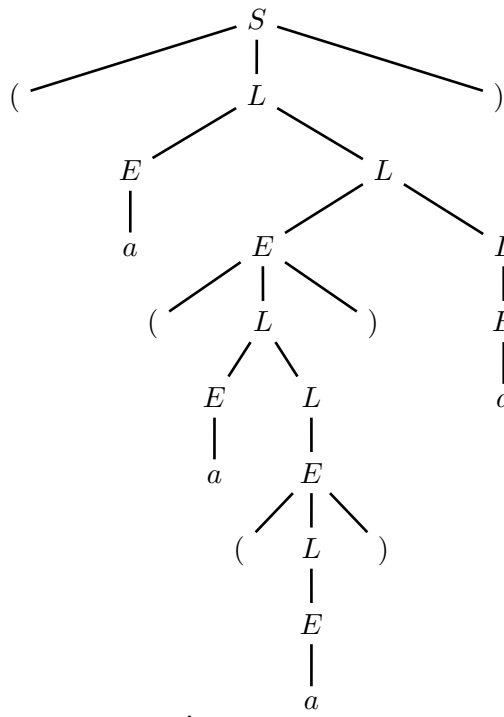
$$2: L_0 \rightarrow E_1 L_2$$

$$3: L_0 \rightarrow E_1$$

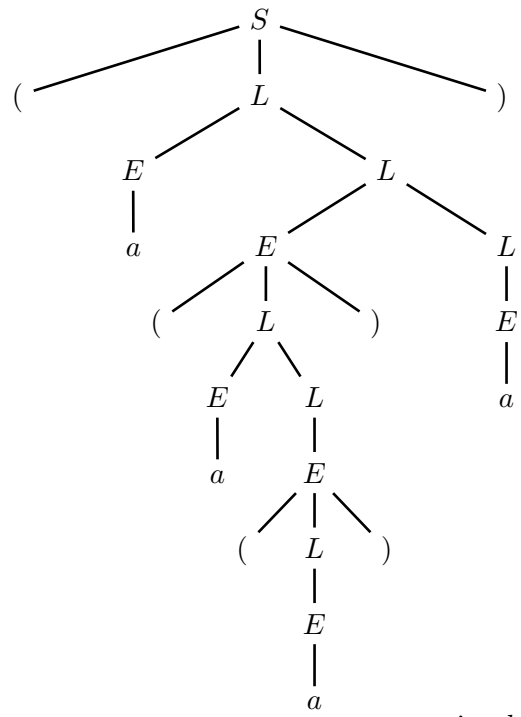
$$4: E_0 \rightarrow a$$

$$5: E_0 \rightarrow (L_1)$$

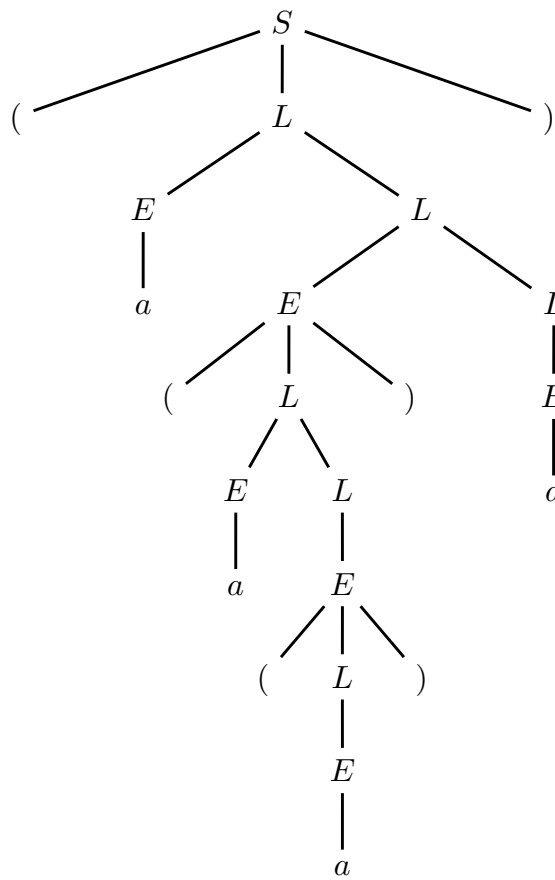
syntax trees to be decorated (one for each question)



question a



question b



question c

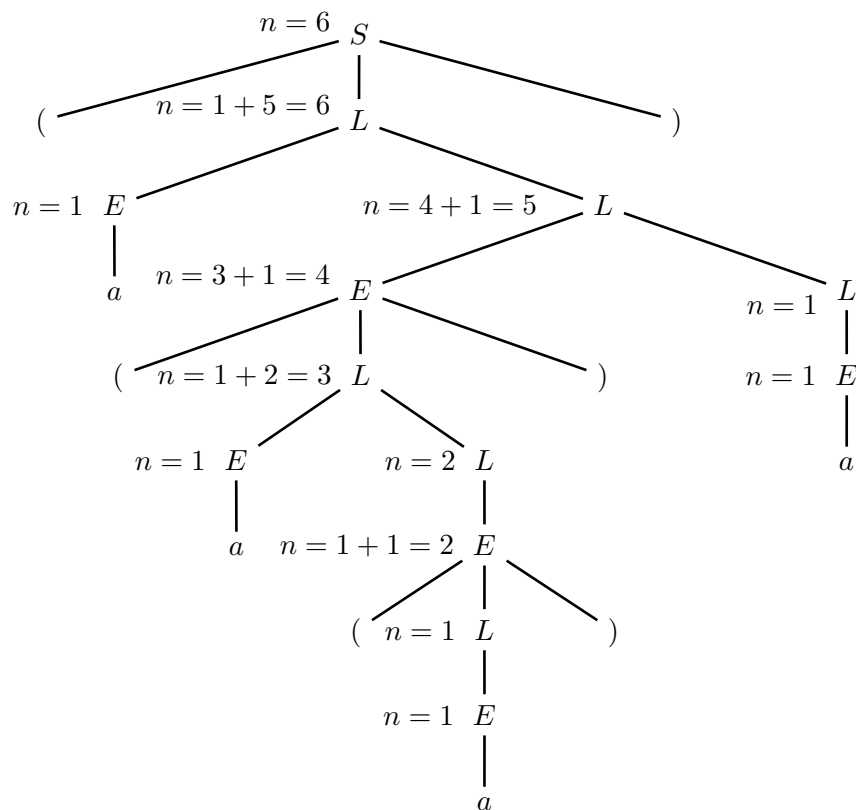
Solution

(a) Here are the left attribute n and its semantic functions:

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
left	n	integer ≥ 1	S, L, E	total number of elements (atoms and sublists)

#	<i>syntax</i>	<i>semantics</i> - question <i>a</i>
1:	$S_0 \rightarrow (L_1)$	$n_0 = n_1$
2:	$L_0 \rightarrow E_1 L_2$	$n_0 = n_1 + n_2$
3:	$L_0 \rightarrow E_1$	$n_0 = n_1$
4:	$E_0 \rightarrow a$	$n_0 = 1$
5:	$E_0 \rightarrow (L_1)$	$n_0 = n_1 + 1$

And here is the syntax tree decorated with the values of the left attribute n :

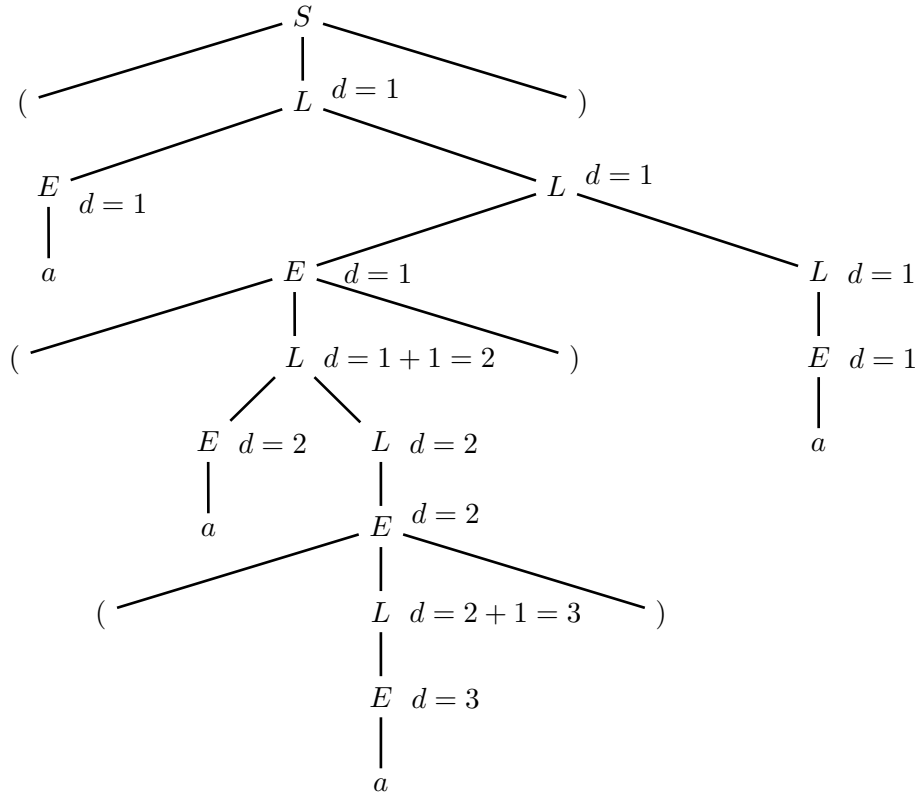


(b) Here are the right attribute d and its semantic functions:

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
right	d	integer ≥ 1	L, E	nesting level of an element (atom or sublist)

#	<i>syntax</i>	<i>semantics</i> - question b
1:	$S_0 \rightarrow (L_1)$	$d_1 = 1$
2:	$L_0 \rightarrow E_1 L_2$	$d_1 = d_0$ $d_2 = d_0$
3:	$L_0 \rightarrow E_1$	$d_1 = d_0$
4:	$E_0 \rightarrow a$	none (or formally $a_1 = d_0$) ⁰
5:	$E_0 \rightarrow (L_1)$	$d_1 = d_0 + 1$

And here is the syntax tree decorated with the values of the right attribute d :



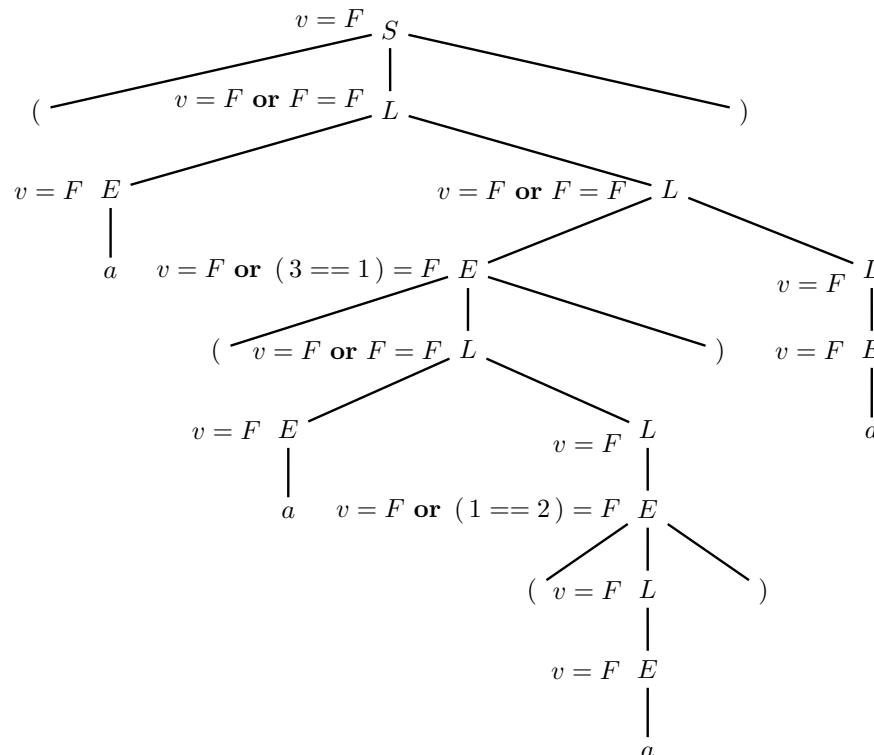
⁰The attribute grammar model allows one to associate a right attribute to a terminal, although here such a position plays a purely formal role and might only be justified for completeness.

(c) Here are the left attribute v and its semantic functions:

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
left	v	boolean	S, L, E	this predicate is true if and only if in the expression there is a proper sublist (i.e., not coincident with the entire expression) that has a total number of elements (atoms and sublists) equal to its nesting level

#	<i>syntax</i>	<i>semantics</i> - question <i>c</i>
1:	$S_0 \rightarrow (L_1)$	$v_0 = v_1$
2:	$L_0 \rightarrow E_1 L_2$	$v_0 = v_1$ or v_2
3:	$L_0 \rightarrow E_1$	$v_0 = v_1$
4:	$E_0 \rightarrow a$	$v_0 = F$
5:	$E_0 \rightarrow (L_1)$	$v_0 = v_1$ or $(n_1 == d_0)$

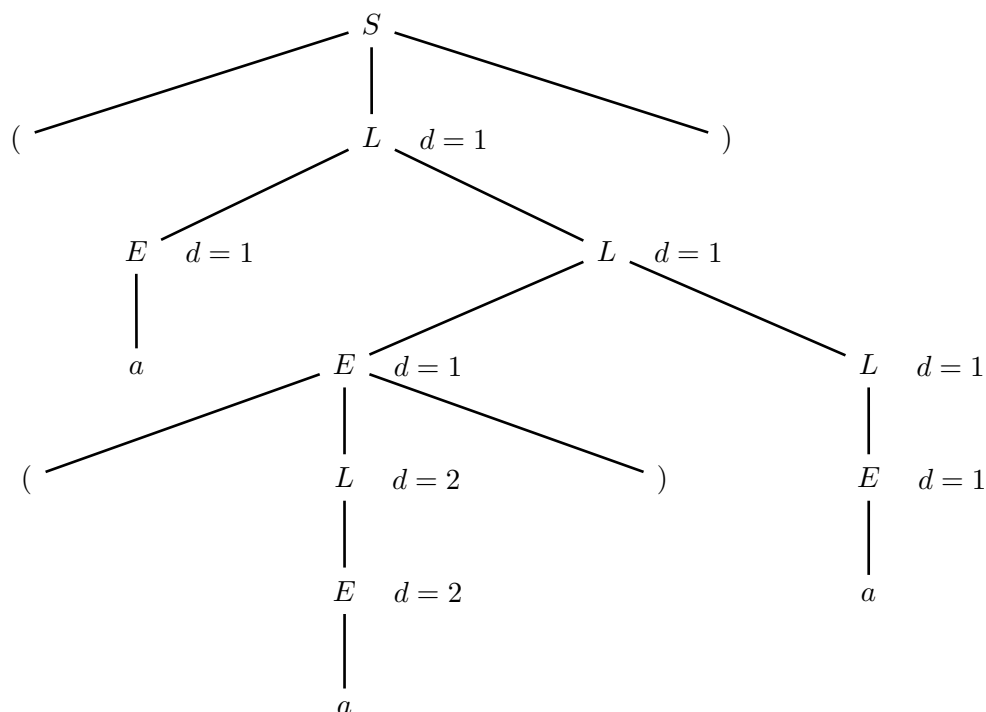
And here is the syntax tree decorated with the values of the left attribute v :

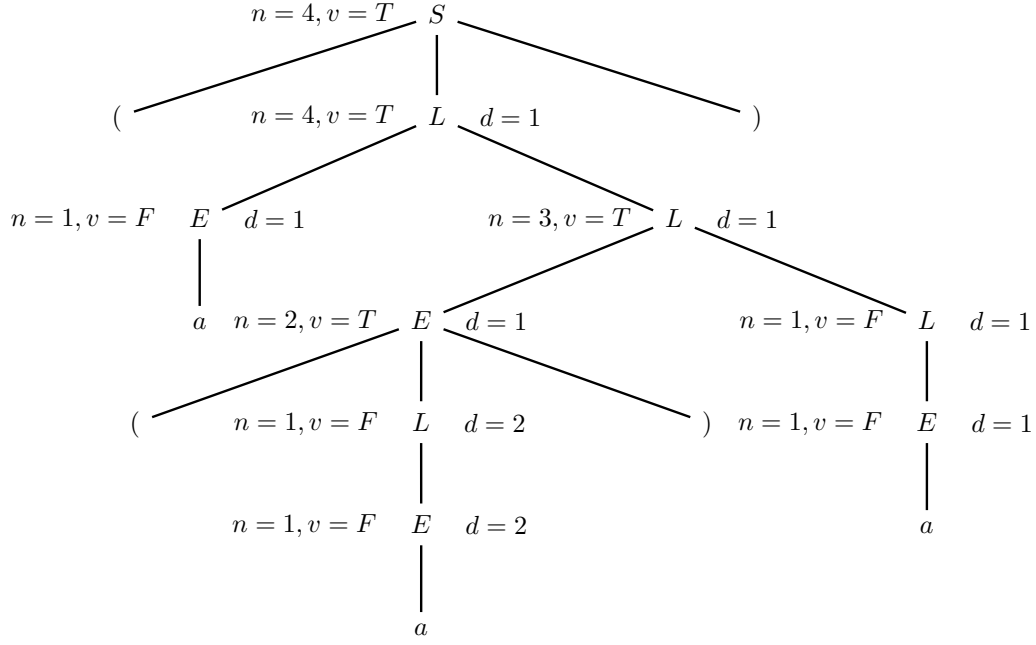


We list here a few additional (not requested) observations. First of all, it is not necessary to add any more auxiliary attributes to the three ones already given. Of course, there may be other solutions that use four or even more attributes.

Second, notice that the full attribute grammar G_a , with all the three attributes and their semantic functions, is of type one-sweep. In fact, the attributes n and d are of type left and right, respectively, they are independent of each other, and the latter (d) depends only on itself in the parent node; and the attribute v is of type left and depends on itself (left) in the child nodes, on attribute n (left) in the child nodes, and on attribute d (right) in the parent node. Thus grammar G_a satisfies the one-sweep condition. Therefore the attributes n , d and v are computable in this order: first d from top to bottom, then n and v altogether from bottom to top (using the values of d already computed). The evaluation order of the child nodes E and L in the rule 2 is free, as their right attributes are independent of each other.

For instance, consider the new expression $e' = (a(a)a)$. In total expression e' has four elements, namely the sublist (a) and three atoms a . The sublist (a) is at level 1 and has one element (the atom a). Therefore the whole expression e' has $v = T$. Here is the one-sweep evaluation (first top-to-bottom and then bottom-to-top):





Third, since the subtree evaluation order in the rule 2 (the only one with two nonterminals in the right part) is free (since attribute d depends only on itself in the parent node), it can be taken left-to-right. Anyway, the *BNF* syntactic support G is not of type $LL(k)$ for any $k \geq 1$. In fact, the guide sets of the two alternative rules 2 and 3 overlap for any $k \geq 1$, as both sets contain a string of k open round brackets “ $((\dots ($ ”, due to the existence of the recursive derivation $E \Rightarrow (L) \Rightarrow (E)$ for the initial nonterminal E of the two rules. Thus the attribute grammar G_a is not of type L and it does not have an integrated recursive descent semantic analyzer.